SESSION 04 / PRINTABLE GUIDE

Notebook 101: Learn the execution model before the main lab

Session 04 | First-time-user warm-up
Output: A small working notebook, an explained error, equivalent PySpark/Spark SQL results, and a safe persistence demonstration.

Complete the pre-delivery prerequisites first. Use an assigned, isolated training Lakehouse, not a production or shared output target. These four in-memory rows teach mechanics, not analytical scale. The main lab still uses the realistic datasets.

How to use the starter and solution

Import the starter notebook using the import instructions. It contains ready-to-run examples and **three short practice cells**. Each practice cell intentionally raises `NotImplementedError` until you replace it with your answer. That is a labeled teaching stop, not a permissions problem. The completed solution fills those cells; the Python blocks in this handout show the same worked answers.

Use the solution for presenter preparation or recovery, not as evidence that the learner completed the exercises. Start a fresh session and run cells in order except when deliberately testing execution order. Both notebooks skip persistent writes by default.

1. Find your way around

Before running anything, identify:

- The notebook name, its PySpark language/kernel, and the assigned workspace.
- The default Lakehouse and its **Files** and **Tables** areas.
- A Markdown cell, a code cell, the run-cell action, and Run all.
- Cell output, errors, and the session/job progress controls available in the tenant.

Explain the objects: a notebook stores code/instructions; a session holds execution state; a DataFrame describes data/computation; a Lakehouse persists files/tables; a Warehouse is a separate SQL serving destination.

Review code before executing it. Run all also runs write cells and practice stops. Saving the notebook is not saving a DataFrame or its variables.

2. Establish explicit configuration

```python
import re
from decimal import Decimal
from pyspark.sql import functions as F, types as T

LEARNER_ID = "pair01"
BATCH_ID = "s04-101-001"
ALLOW_TRAINING_WRITE = False
VERIFY_PERSISTED_TABLE = False

if not re.fullmatch(r"[a-z][a-z0-9_]{0,19}", LEARNER_ID):
    raise ValueError("Use the facilitator-assigned lowercase learner/pair ID.")
OUTPUT_TABLE = f"s04_101_{LEARNER_ID}_demo"
spark.conf.set("spark.sql.session.timeZone", "UTC")
print("Spark version:", spark.version)
print("Catalog/schema context:", spark.catalog.currentDatabase())
print("Proposed training table:", OUTPUT_TABLE)
```

Set `LEARNER_ID` to the facilitator-assigned value. A printed catalog/schema is helpful context, **not proof** of the selected Lakehouse: verify the default attachment in the UI with the facilitator before any write.

The flags are ordinary Python variables, not Fabric pipeline parameters. Keep configuration together so a reviewer can see the input/output intent. Do not put passwords, account keys, or tokens in this cell.

3. Experience execution order and stale state

Run the next three cells in order:

```python
warmup_multiplier = 2
```

```python
warmup_result = 10 * warmup_multiplier
```

```python
print("Computed result:", warmup_result)
```

Expect 20. Now edit the first cell to use 3 **without running it** and rerun the calculation: the session still holds 2, so the result remains 20. Run only the assignment cell: displaying `warmup_result` still shows the previously computed 20. Rerun the calculation and display cells to obtain 30.

In a genuinely fresh session, running the result/display cell before its prerequisites produces `NameError`. Previously saved output may remain visible even though the current session has not recreated its variables. Clearing outputs does not delete persisted tables or reset every runtime variable.

Restore the multiplier to 2 when comparing the unmodified worked examples. Restarting the session clears ordinary Python variables and session temporary views; it does not delete Lakehouse Files or Delta tables.

4. Learn only the Python the lab needs

PYTHON

```python
preview_columns = ["device_id", "temperature_c"]
limits = {"minimum_c": Decimal("-40"), "maximum_c": Decimal("85")}
print("First requested column:", preview_columns[0])
print("Approved minimum:", limits["minimum_c"])
```

Strings are text; lists hold ordered items; dictionaries map names to values. Imports make APIs available. Indentation defines Python blocks. A function names reusable logic; its inputs and returned value should be explicit.

Practice 1: Write a small Python function that trims/uppercases a single identifier and returns `None` for blank or missing input. This operates on one ordinary Python value; it is **not** a Spark UDF or a distributed transformation.

<!-- practice: python-normalize -->

PYTHON

```python
def normalize_identifier(value):
    if value is None or not value.strip():
        return None
    return value.strip().upper()


assert normalize_identifier(" d01 ") == "D01"
assert normalize_identifier(" ") is None
assert normalize_identifier(None) is None
```

5. Build a small Spark DataFrame

PYTHON

```python
toy_schema = T.StructType([
    T.StructField("device_id", T.StringType()),
    T.StructField("event_id", T.StringType()),
    T.StructField("temperature_text", T.StringType()),
])
toy_raw = spark.createDataFrame([
    (" d01 ", "E001", "20.00"),
    ("D01", "E002", "22.00"),
    ("D02", "E003", "30.00"),
    ("D02", "E004", "sensor error"),
], toy_schema)
toy_raw.printSchema()
toy_raw.show(truncate=False)
```

`spark` is supplied by the Fabric Spark session. A Spark Column represents an expression, not the current row's Python value. Use `F.upper(F.col("device_id"))`, not a Python scalar function applied directly to a Spark Column.

This table has four rows, three source string columns, and one unusable numeric value. Its simplicity is intentional: predict every transformation before moving to the much larger main dataset.

Authoring skills: Find help instead of memorizing methods

Add a Markdown cell above the preview with a short question, source scope, and expected count. Add a code cell below it, run only that cell, then move it to the appropriate place without breaking dependencies. Keep a clear distinction between editing code and executing it.

After creating `toy_raw`, type `toy_raw.` or `F.` in an editable code cell and inspect the tenant's available completion suggestions; do not run the incomplete expression. Use the notebook's completion/help controls rather than relying on a keyboard shortcut that may differ by browser/OS. Inspect the documentation for one operation:

PYTHON

```python
help(F.trim)
```

Find its input and result, then explain why a Spark expression differs from the ordinary Python string function in Practice 1. Keep useful Markdown, remove scratch/incomplete expressions, and review saved outputs before sharing.

6. Transform, trigger work, and validate

Practice 2: Normalize device IDs with built-in Spark expressions and create a decimal temperature using tolerant conversion. Preserve the raw text.

<!-- practice: spark-clean -->

```python
toy_clean = (
    toy_raw.withColumn("device_id", F.upper(F.trim("device_id")))
    .withColumn("temperature_c", F.expr(
        "try_cast(temperature_text as decimal(8,2))"
    ))
)
```

Assigning `toy_clean` builds a computation plan; it does not persist a table. Actions such as `show()`, `count()`, and writes execute work. DataFrame transformations normally return new DataFrames, not mutated source objects.

```python
toy_clean.show(truncate=False)
assert "temperature_c" not in toy_raw.columns

toy_accepted = toy_clean.filter(F.col("temperature_c").isNotNull()).select(
    "device_id", "event_id", "temperature_c"
).withColumn("batch_id", F.lit(BATCH_ID))
toy_rejected = toy_clean.filter(F.col("temperature_c").isNull())
assert toy_raw.count() == 4
assert toy_accepted.count() == 3
assert toy_rejected.count() == 1
```

This warm-up checks numeric usability only, **not the main lab's complete key/time/range/context policy**. Missing/unparseable temperature is not zero. Do not silently copy this partial acceptance rule into the realistic workflow.

Notice session startup separately from job execution. Use cell/job progress to distinguish waiting for compute from running a calculation. Follow customer guidance on cancellation and ending your own session; never stop another participant's compute. Avoid collecting raw datasets to the driver just to print or loop through them. These tiny teaching counts are not a recommendation to repeatedly scan a large production batch.

7. Use Spark SQL as a bridge

```python
pyspark_daily = toy_accepted.groupBy("device_id").agg(
    F.count("*").alias("reading_count"),
    F.avg("temperature_c").cast("decimal(8,2)").alias("avg_temperature_c"),
)
toy_accepted.createOrReplaceTempView("s04_101_accepted")
```

Practice 3: Use Spark SQL to return the same device/count/average columns. `spark.sql` below executes **Spark SQL**, not Warehouse T-SQL. The view is session-scoped and does not create a durable Lakehouse or Warehouse table.

<!-- practice: spark-sql -->

```python
sql_daily = spark.sql("""
    SELECT device_id,
           COUNT(*) AS reading_count,
           CAST(AVG(temperature_c) AS DECIMAL(8,2)) AS avg_temperature_c
    FROM s04_101_accepted
    GROUP BY device_id
""")
```

```python
columns = ["device_id", "reading_count", "avg_temperature_c"]
assert pyspark_daily.select(*columns).exceptAll(
    sql_daily.select(*columns)
).limit(1).count() == 0
assert sql_daily.select(*columns).exceptAll(
    pyspark_daily.select(*columns)
).limit(1).count() == 0
display(sql_daily.orderBy("device_id"))
```

Expect D01: two readings, average 21.00; D02: one reading, average 30.00. Matching a preview is weaker than comparing both complete small results.

8. Persist safely, then prove what survives a restart

Before the optional write, confirm the approved default Lakehouse and proposed learner-specific target with the facilitator. Only then change `ALLOW_TRAINING_WRITE` to `True` in the configuration cell and rerun that cell. The following cell creates **one new Delta table**; it never appends, overwrites, drops, or publishes Warehouse data.

PYTHON

```python
if ALLOW_TRAINING_WRITE:
    toy_accepted.write.format("delta").mode("errorifexists").saveAsTable(OUTPUT_TABLE)
    print("Created:", OUTPUT_TABLE)
else:
    print("Write skipped. Facilitator approval is required before enabling it.")
```

An unchanged second write intentionally fails because the target exists. Do not change the write mode just to get a green cell. For another authorized creation, the facilitator must assign a new learner/run target.

Write mode	Meaning	Beginner safety lesson
`errorifexists`	Refuse an existing target	Protect an unknown prior result
`append`	Add records	Repeated runs can duplicate data
`overwrite`	Replace a target snapshot	Use only an explicitly approved isolated target

After a successful optional creation, save the notebook. Stop/restart your own session using the tenant's session controls. Notice that variables/temp views must be recreated. In the configuration cell keep `ALLOW_TRAINING_WRITE=False` and set `VERIFY_PERSISTED_TABLE=True`. Run the completed notebook from the top without re-enabling the write, or rerun the preceding cells in order.

PYTHON

```python
if VERIFY_PERSISTED_TABLE:
    persisted = spark.table(OUTPUT_TABLE)
    fields = ["device_id", "event_id", "temperature_c", "batch_id"]
    assert persisted.select(*fields).exceptAll(
        toy_accepted.select(*fields)
    ).limit(1).count() == 0
    assert toy_accepted.select(*fields).exceptAll(
        persisted.select(*fields)
    ).limit(1).count() == 0
    display(persisted.orderBy("device_id", "event_id"))
else:
    print("Read-back skipped; enable it only after the approved table was created.")
```

The three accepted rows survive a session restart because they were persisted. The notebook's saved code/output is not that persistence mechanism. Read-back does not establish that a Warehouse Gold table exists.

Inspect the output without changing its storage

If the optional write was approved, locate `OUTPUT_TABLE` in the Lakehouse Tables view and preview the three rows. If the tenant provides an approved underlying-file view, inspect it read-only; otherwise use presenter-prepared evidence or revisit this after the main lab's Delta tables exist.

Explain the difference between the **logical table**, its data files, and its Delta transaction log. Never manually rename, edit, or delete files underneath the table. Spark `.write.csv(...)` would normally create an output directory with part files, not necessarily one neatly named CSV. That is different from the unchanged one-file-per-export Binary copies used in pre-delivery setup. Do not add an extra export or force `coalesce(1)` just to make the output look like a desktop file.

9. Fix a broken notebook

Import the troubleshooting starter into a separate notebook with the prepared default Lakehouse. It reads only the first daily scenario CSV and has **three deliberate errors**:

Error	Recognize	Repair
Missing variable	`NameError`; defining cell has not run or does not exist	Define the preview limit before using it
Misspelled column	Spark cannot resolve `temperatur_c`	Inspect the schema and use `temperature_c`
Wrong source folder	Path-not-found error for `devicez`	Compare with the prepared `devices` folder and correct the path

Run setup first, then work on one error cell at a time. Read the first useful exception message; identify what changed and why the repair is valid. Do not guess new credentials, change the customer's permissions, or weaken network controls. An unexpected setup/access failure is a facilitator readiness issue, not one of the intentional mistakes.

After fixing all three, restart your own session and run the repaired notebook in order. All cells should succeed without a write. Use the completed troubleshooting solution for comparison or recovery. A green solution run is not proof of learner repair.

Completion evidence and transition

Where am I? Paths, names, and permission boundaries

Use this map before troubleshooting a read:

Reference	What it means in this lesson
`Files/session_04/scenario/devices/*.csv`	Spark-relative path in the **default Lakehouse**, not a laptop folder

Reference	What it means in this lesson
Explicit OneLake path copied from the UI	Resource-specific address; check which workspace/item it names and what path type was copied
Approved ADLS DFS endpoint + container/prefix	External source for the prepared file-copy pipeline
`s04_lab_device_silver`	Spark table resolved using the current Lakehouse/ catalog/schema context
`gold.s04_lab_device_daily`	Warehouse SQL object, not automatically readable as the same named Spark table

Locate the first scenario CSV in the prepared Lakehouse Files area. Inspect the available copy-path options and compare a Spark-relative path with an explicit resource path. Read the correct file during the repair exercise; do not change the default attachment merely to make an unknown target resolve. Browser URLs, local laptop paths, and an external ADLS endpoint are not interchangeable with the notebook's `Files/...` path.

Explain three separate boundaries: permission to open/edit the notebook; permission for its actual execution identity to read/write the Lakehouse or use a source connection; and permission to query/ publish Warehouse objects. An ADLS copy can use a different approved identity from an interactive notebook. Seeing an item or owning a notebook does not grant all those data permissions. If access fails, report the resource, identity context, useful error, and run reference to the facilitator; do not expose credentials or grant yourself broad roles. Do not deliberately break customer permissions for this exercise.

Keep the survival guide beside the notebook. Use its decision table to classify a deliberate repair error and one presenter-provided access/session example. Distinguish expected task/safety stops from actual environment failures; check session startup/job progress before blindly retrying a cell that could write.

Explain what you completed

The learner should be able to explain stale state versus saved output, identify Python versus Spark expressions, reproduce the three accepted toy events and two equivalent summaries, distinguish Spark SQL from Warehouse SQL, and explain the write flags and all three repairs.

If persistence is demonstrated, also retain the approved target name and successful restart/read-back evidence; otherwise label that step skipped. No cleanup or target overwrite is implicit.

Next: Review when notebooks are justified, then import the main-lab starter. The main-lab solution contains worked practice answers; both use the same validated scenario pipeline and an explicit write-approval gate. The PySpark reference is a lookup aid, not a vocabulary list to memorize. After the main lab, attempt the independent challenge without a step-by-step answer.

Reference: Microsoft Fabric notebook usage. UI labels and feature availability require the actual tenant preflight.

Source: 00_notebook_101.md. Companion-material links open the repository originals.